Deep Learning for teens

By Hamid Sadeghi

Table of contents

ANN vs NN

- **NN (Neural Network)** → general term for any model made of connected artificial neurons.
- **ANN (Artificial Neural Network)** → the basic feedforward NN (input → hidden → output), used for regression/classification.
- Every ANN is an NN, but not every NN is an ANN.

Dense Layer(fully connected layer)

- **Dense Layer**
  - **every neuron is connected to every neuron in the next layer.**

Activation Function

- Each neuron takes in inputs, multiplies them by its weights, adds a bias, and then passes the result through an **activation function**.

Activation Function

Function	Output Range	Typical Use	Notes
Sigmoid	$0 \rightarrow 1$	Binary classification output layer	Smooth curve, can saturate
Tanh	$-1 \rightarrow 1$	Hidden layers (older models)	Centered output, saturation issue
ReLU	$0 \rightarrow \infty$	Most hidden layers	Fast, simple, standard choice
Leaky ReLU	Negative slope for $x<0$	Hidden layers	Fixes dying ReLU problem
Parametric ReLU (PReLU)	Learnable slope	Hidden layers	Slope learned during training
ELU	Negative exponential for $x<0$	Hidden layers	Smooths learning, better than ReLU
Swish	$0 \rightarrow \infty$	Hidden layers (Google models)	Self-gated, smoother than ReLU

Dense Layer + Activation Function Choices

- **Hidden Dense Layers**
  - Most common: **ReLU**
    - Fast, simple, avoids saturation.
    - Default choice for hidden layers.
  - Alternatives: **Leaky ReLU**, **ELU**, **Swish**, **GELU**
    - Used when you want smoother learning or to fix “dying ReLU” problems.
- **Output Dense Layer**
  - **Sigmoid** → for **binary classification** (output between 0 and 1).
  - **Softmax** → for **multi-class classification** (outputs probabilities across classes).

optimizer

- In deep learning, an optimizer is the algorithm that updates the weights of your network to minimize the loss function

optimizer

Optimizer	Key Idea	Strengths	Weaknesses
SGD (Stochastic Gradient Descent)	Updates weights using one/few samples at a time	Simple, widely used, good generalization	Can be slow, sensitive to learning rate
Momentum	Adds “memory” of past gradients	Faster convergence, avoids local minima	Needs tuning of momentum parameter
Adagrad	Adaptive learning rate per parameter	Good for sparse data	Learning rate shrinks too much over time
RMSProp	Keeps moving average of squared gradients	Works well for RNNs, stabilizes training	Hyperparameters need tuning
Adam	Combines Momentum + RMSProp	Fast, robust, default choice in many cases	Can overfit, sometimes less generalization
Nadam	Adam + Nesterov momentum	Often faster convergence	More complex, extra tuning needed

Types of Optimizers and Their Use Cases

- Classification (CNNs, vision) → SGD with Momentum or Adam.
- NLP / Sparse features → Adagrad or Adam.
- RNNs / sequential tasks → RMSProp or Adam.
- General purpose / default → Adam.

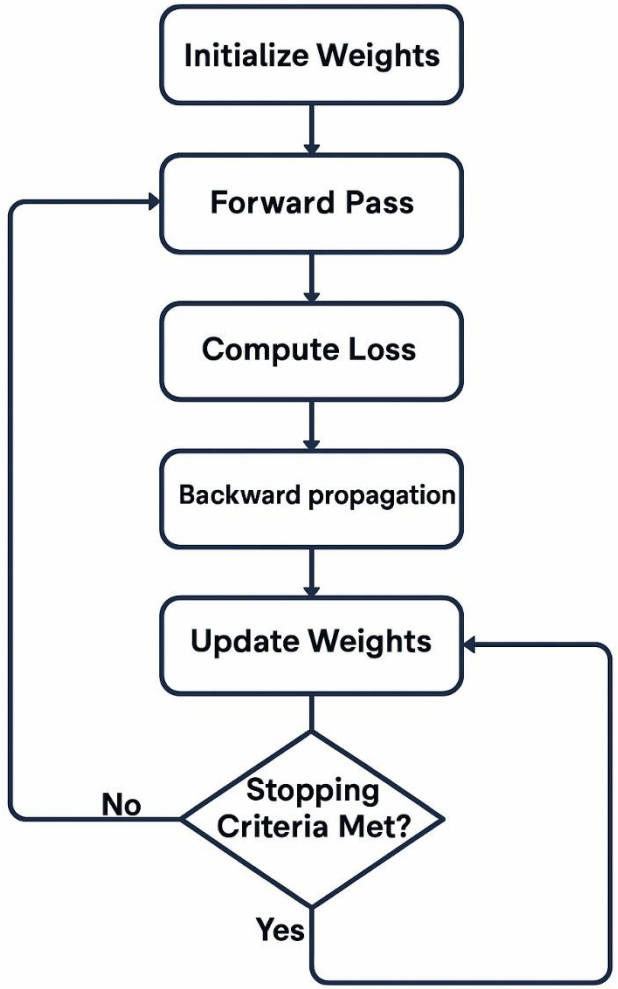
Batch gradient decent, Stochastic gradient decent vs Mini batch gradient decent

- Batch gradient decent
  - We go through **all** the sample in dataset
    - For large dataset link several millions records that's so hard
- Stochastic gradient decent
  - Randomly pick **one** sample(record)
- Mini batch gradient decent
  - We use several(batch) samples not all or one per epoch

Forward Pass Breakdown

- **Initialize inputs**: Start with your input features X
- **Weighted sum:** Each neuron computes z=w.x+b (weights times inputs plus bias).
- **Activation function**: Apply a non-linear function (like ReLU, sigmoid, tanh) to z
- **Layer by layer propagation**: Pass activations forward through hidden layers until reaching the output layer.
- **Final output**: The network produces y_predicted
- **Loss calculation**: Compare

Forward Pass Breakdown



Loss

- Difference between actual y and y predicted
- Types of loss
  - Regression → MSE, MAE, Huber
  - Binary classification → Binary Cross-Entropy
  - Multi-class classification → Categorical Cross-Entropy (or Sparse version)
  - Margin-based models (SVM) → Hinge Loss
  - Probabilistic models → KL Divergence
  - PyTorch classification → NLLLoss

metric

it’s just a way to measure how well your model is performing

- **Accuracy**
- Precision
- Recall
- F1-score
- **Loss**

batch_size

- batch_size simply means how many training samples the model processes before updating its weights once.

Homework

- Use the Diabetes dataset in scikit-learn.
- Build an ANN with an appropriate number of layers. Select the right loss function, activation function, metric, and optimizer.